Hermite Integrator for a Gravitational N-Body System

Zachary Coustier
May 2025

I. ABSTRACT

This paper discusses my efforts to implement a Hermite Integration scheme to a gravitational N-Body system. It gives background on the math involved alongside some of the early papers that discussed implementing this scheme for this task. Some of the code is also shown alongside some comments explaining it. Finally, resulting plots are shown for several values of N as well as different starting velocities and positions.

II. INTRODUCTION

One popular method for numerical integration is the Runge-Kutta method, which we learned about earlier this semester. I was curious about finding what other kinds of methods were out there, and how to apply it to an orbital N-body simulation. An example of another method was used in a 2010 paper by Hurley [1]. In this paper they conducted N-body simulations of astronomical objects known as extended star clusters to help provide theories on the origin of their existence.

The method used in that paper was known as a Hermite Integrator Scheme. This type of method caught my eye because although it is a fourth-order method, it works with the derivative of accleration, also known as *jerk*. I was curious to see how to put this method in practice, and how feasible it was to implement it when comparing to the previous methods we learned about this semester. This paper will show the background, some of the code, and results that came out of using the Hermite method on an N-body system only under the influence of gravity.

III. MODEL

The best place to begin with this method is a 1992 paper that includes Sverre J. Aarseth as one of the two authors [2]. He developed programs using it dating back to the decades before this paper, as well as outlining the background theory here as well. We first begin with a formula used in order to move further, for both the position and velocities.

$$x_{p,j} = \frac{(t-t_j)^3}{6}\dot{a_j} + \frac{(t-t_j)^2}{2}a_j + (t-t_j)v_j + x_j \tag{1}$$

and

$$v_{p,j} = \frac{(t-t_j)^2}{2}\dot{a_j} + (t-t_j)a_j + v_j \tag{2}$$

Here, the current particle is the ith particle, while the index j runs through all particles (including i). One strength of this method mentioned in the same paper is that because the predictor of the Hermite Individual Timestep Scheme (HITS) does not require any information about previous timesteps, it is self-starting. Using these first two equations, we can arrive at calculations for both the acceleration and the jerk:

$$a_i = Gm\frac{r_{ij}}{(r_{ij}^2+\epsilon^2)^{3/2}} \tag{3}$$

as well as

$$\dot{a_i} = Gm[\frac{r_{ij}}{(r_{ij}^2 + \epsilon^2)^{3/2}} + \frac{3(v_{ij} \cdot r_{ij})r_{ij}}{(r_{ij}^2 + \epsilon^2)^{5/2}}] \tag{4}$$

Here, ϵ is a softening parameter. The last set of equations to look at are the prediction equations I used for both the position and the velocity:

$$r_{i,pred} = r_{i,0} + v_{i,0}\Delta t_i + \frac{1}{2!}a_{i,0}\Delta t_i^2 + \frac{1}{3!}\dot{a}_{i,0}\Delta t_i^3 \tag{5}$$

$$v_{i,pred} = v_{i,0} + v_{i,0}\Delta t_i + \frac{1}{2!}v_{i,0}\Delta t_i^2 \tag{6}$$

Another 1999 paper by Aarseth goes into more technical details on this scheme as well as other schemes [3]. I was curious and came across another paper that delves into even further detail, as well as higher order derivatives of acceleration (humorously named *snap* and *crackle*), in a 2008 short paper on the subject [4]. They propose that the increased computing power for this method is not much higher, in exchange for better accuracy. This may be a place for future work to be pursued.

IV. METHOD

In order to pursue this, I used JupyterNotebook to run Python code. The libraries used include numpy for the calculation, and matplotlib for the plotting and animation aspects. I chose to work with easier units, with time in years, distance in Astronomical Units (AU), and mass in Solar Mass. I set each N-body to a mass of 1, with a central mass equal to 1000 times that. The initial parameters I tweaked with was the number of bodies in the code, the number of steps, as well as my step in time. After this initialization, I created three functions: One function to create the body objects themselves, one function to calculate acceleration and jerk, and finally one to implement the Hermite method. In my function to create my N-bodies, I included some randomness in both their angular and radial position, within limits. I can also set the initial velocities in each direction to zero to arrive at a simulation that is slower to begin with.

Then, after the accleration and jerk function, I arrive at the Hermite function. This function includes the equations I arrived at in the Model section of the predicted position and velocity. I also need initial values, which come in the form of initial acceleration, jerk, as well as arrays for positions, velocities, and masses. I then predict the next acceleration and jerk based off the predicted position and velocity before corrections for position and velocity. These corrections use the acceleration and jerk just obtained. I then create a new bodies array that takes these into consideration for use later.

```
def hermite_step(bodies, dt):
    n = len(bodies)

    # Current state of pos, velo
    positions = np.array([[i[0], i[1], i[2]] for i in bodies])
    velocities = np.array([[i[3], i[4], i[5]] for i in bodies])
    masses = np.array([i[6] for i in bodies])

    # a0 and j0
```

```
    acc, jerk = calculate_acc_and_jerk(positions, velocities, masses)

    # predictions
    pred_positions = (positions + velocities * dt
                      + 0.5 * acc * dt**2 + (1/6) * jerk * dt**3)
    pred_velocities = velocities + acc * dt + 0.5 * jerk * dt**2


    # a1 and j1 (predictions)
    acc_p, jerk_p = compute_acc_and_jerk(pred_positions, pred_velocities, masses)

    # correction for pos, velo
    new_positions = (positions + 0.5 * (velocities + pred_velocities) * dt
                     + (1/12) * (acc_p - acc) * dt**2)
    new_velocities = (velocities + 0.5 * (acc + acc_p) * dt
                      + (1/12) * (jerk_p - jerk) * dt**2)


    # update the bodies
    new_bodies = []
    for i in range(n):
        x, y, z = new_positions[i]
        vx, vy, vz = new_velocities[i]
        m = masses[i]
        new_bodies.append((x, y, z, vx, vy, vz, m))

    return new_bodies
```

After these three functions are created, I can then call them to create an array of trajectories as well as time evolution in order to plot and animate the results. I tried a variety of number of objects, as well as setting their initial velocity to 0 as well as slighlty randomizing their starting velocities.

V. RESULTS

I was able to successfully implement the Hermite scheme in order to create both plots and animations. On my machine, I could go up to about N=200 before it would stop running. Using stronger machines could allow for this scheme to go up to much higher values of N if needed. The values of number of steps along with time step size I used gave good results in the animations. The resulting animations will be separate from this. Here are a couple examples of the resulting plots, with the black dots being each objects starting position:

VI. CONCLUSION

I was able to successfully implement a Hermite integration scheme to a gravitational N-body system. They were plotted in 2 dimensional space, as well as animated. Where I left off in this work allows for future inquiries to be pursued. A direct comparison to Runge-Kutta, finding hardware

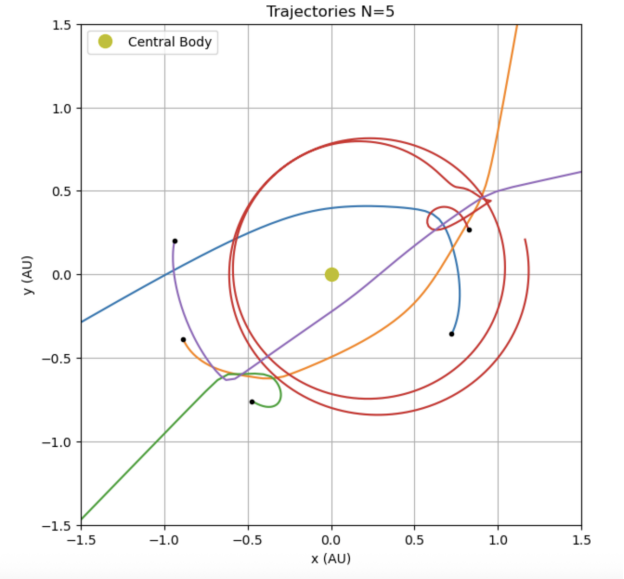


FIG. 1: N=5 with no starting velocity.

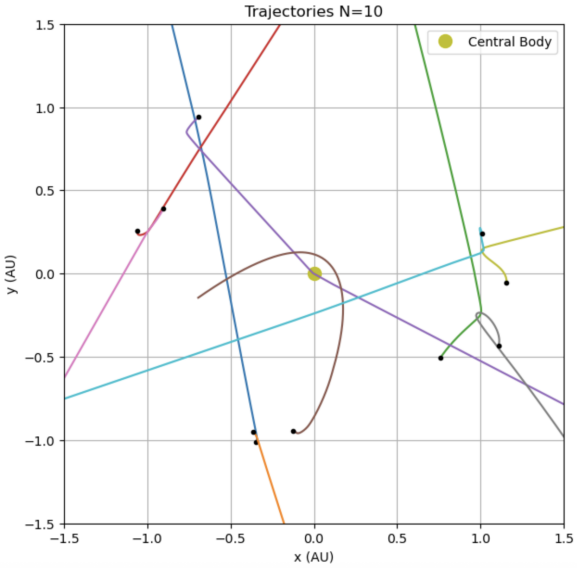


FIG. 2: N=10 with no starting velocity.

to allow for N to go above 1000, as well as trying out even higher order integration schemes are all areas for future exploration. Another area for the 4-th order Hermite scheme that could be attempted is different correction steps, as mentioned in Aarseth's 1992 paper, which has different coefficients. One final step would be to recreate the plot in the 2008 paper that compares error and efficiency of higher order methods [4].

[1] Jarrod R. Hurley and A. Dougal Mackey, "N-body models of extended star clusters," **408**, 2353–2363 (Nov. 2010), arXiv:1008.4991 [astro-ph.GA]

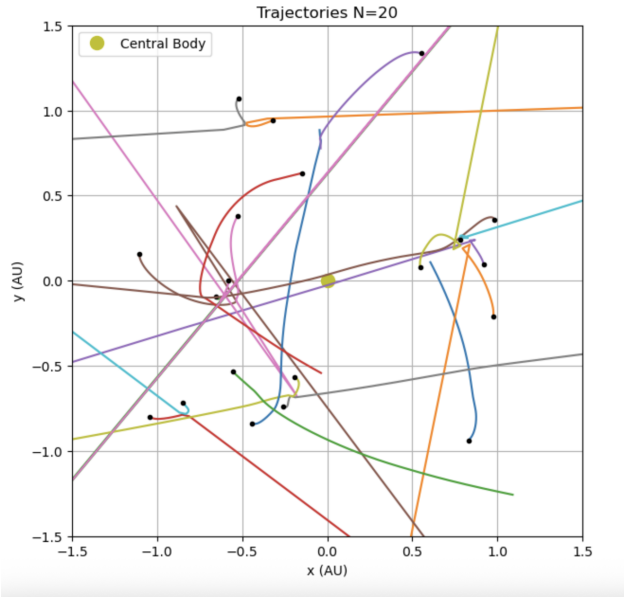


FIG. 3: N=20 with randomized starting velocity. This run also contained more variation in the starting radial distance for each object.

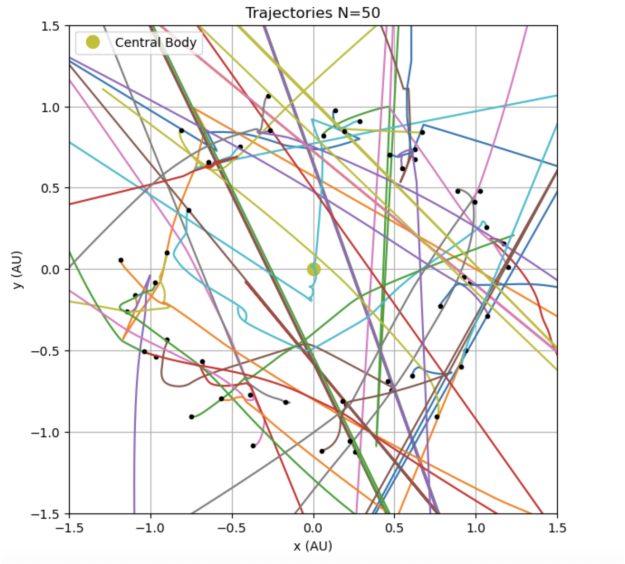


FIG. 4: N=50 with randomized starting velocity.

[2] Junichiro Makino and Sverre J. Aarseth, "On a Hermite Integrator with Ahmad-Cohen Scheme for Gravitational Many-Body Problems," **44**, 141–151 (Apr. 1992)

[3] Sverre J. Aarseth, "From NBODY1 to NBODY6: The Growth of an Industry," **111**, 1333–1346 (Nov. 1999)

[4] Keigo Nitadori, Masaki Iwasawa, and Junichiro Makino, "6th and 8th Order Hermite Integrator Using Snap and Crackle," in *Dynamical Evolution of Dense Stellar Systems*, IAU Symposium, Vol. 246, edited by Enrico Vesperini, Mirek Giersz, and Alison Sills (2008) pp. 473–474

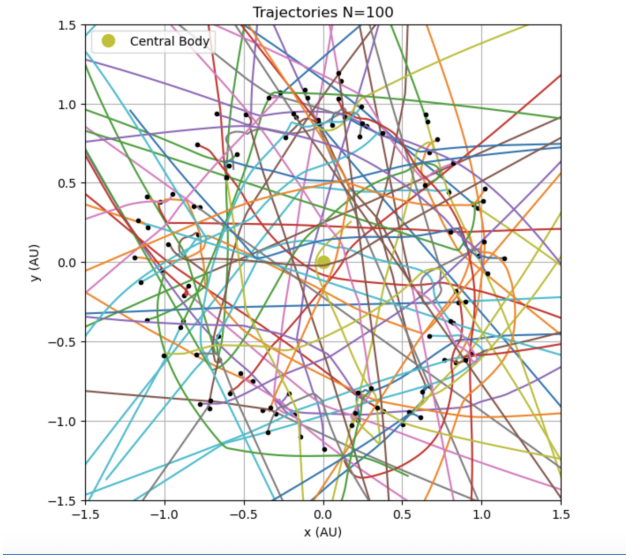


FIG. 5: N=100 with randomized starting velocity.